Driver Monitoring System

Assignment 3 — Autonomous Vehicles

Carlo Squarcia Mateo — Erasmus Student

2024/2025

1. Introduction

Driver distraction and fatigue are two of the main causes of traffic accidents in the world. Modern vehicles are starting to include systems that monitor the driver in real time to detect dangerous situations before they cause an accident. These systems are called Driver Monitoring Systems (DMS).

The goal of this assignment is to build a functional DMS using only a standard laptop webcam and open-source software. The system must be able to detect whether the driver is paying attention to the road, identify signs of distraction or fatigue, estimate the heart rate without any physical sensors, and show all this information in real time on the video feed.

The system was implemented in Python 3 using OpenCV for video capture, MediaPipe for facial landmark detection, and several custom modules for the different detection tasks.

2. System Architecture

The system is structured as a pipeline where each frame from the webcam passes through a series of processing steps. The diagram below shows how the different modules connect to each other.

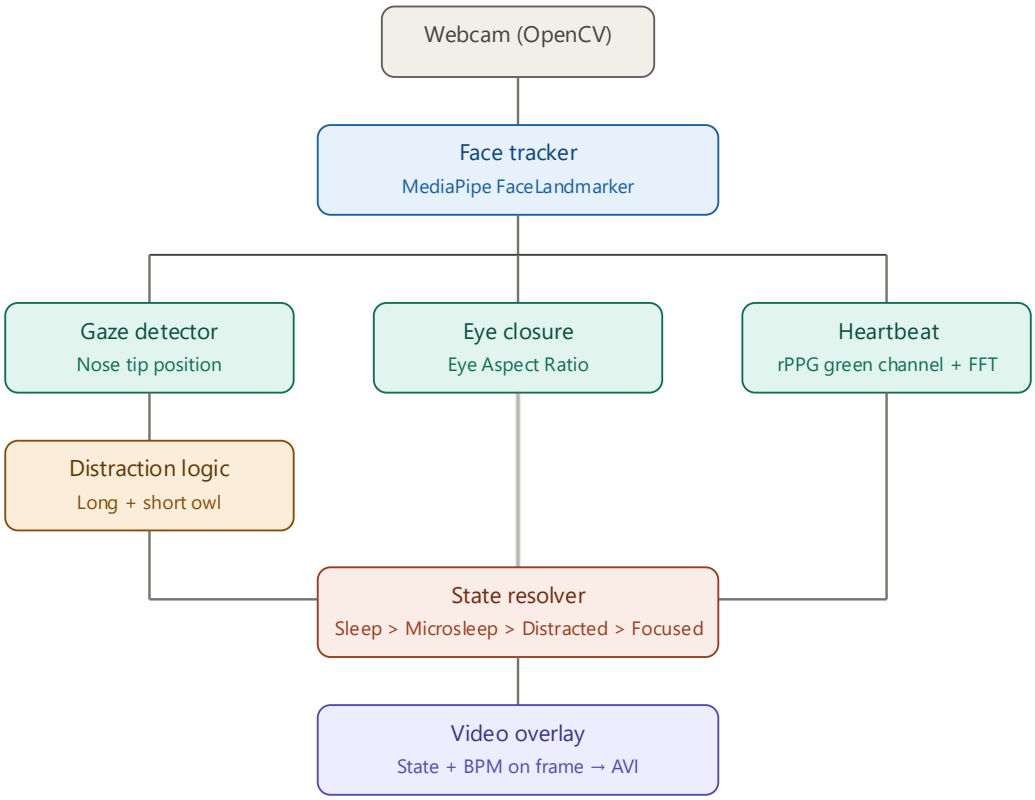

Figure: System pipeline block diagram

The main components of the system are:

- Face Tracker — detects the face and extracts 478 facial landmarks using MediaPipe.
- Gaze Detector — estimates whether the driver is looking at the road or away, using the nose tip position.
- Distraction Logic — decides if the driver is distracted based on how long and how often they look away.
- Eye Closure Detector — measures if the eyes are closed and for how long, to detect microsleep and sleep.
- Heartbeat Estimator — estimates the heart rate from small colour changes in the forehead skin.
- Video Overlay — draws the current state and heart rate on the video frame.

The project follows a modular structure where each component is in a separate Python file inside the dms/ folder. The main file runDMS.py initialises all modules and runs the processing loop.

3. Implementation

3.1 Face Detection

Facial landmarks are detected using MediaPipe FaceLandmarker, which is the new Tasks API introduced in MediaPipe 0.10. The old mp.solutions.face_mesh interface was removed in recent versions, so this project uses the newer API which requires downloading a separate model file (face_landmarker.task).

The model produces 478 3D landmarks for each face. From these, the system uses a small subset: the nose tip (landmark 4), six landmarks around each eye for the Eye Aspect Ratio calculation, and the iris centres (landmarks 468 and 473) for optional gaze refinement. If no face is detected in a frame, the system shows a "Face not detected" warning and continues running normally.

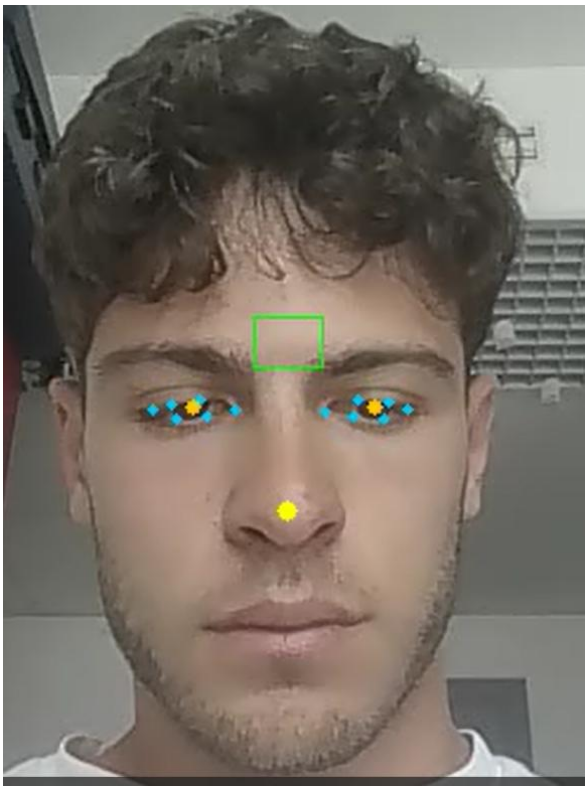

Figure: System running — face detected with landmarks visible

3.2 Gaze Estimation and Calibration

Instead of using a full 3D head pose estimation, which would require complex mathematical operations, the system uses a simpler approach based on the horizontal position of the nose tip. When the driver looks left or right, the nose moves horizontally across the frame in a very consistent way.

At the start of the program, there is a calibration phase of 3 seconds. During this time, the driver must look straight ahead. The system records the average horizontal position of the nose tip as the reference value. After calibration, if the nose deviates more than 8% of the frame width from this reference, the driver is considered to be looking away from the road.

This approach is simple and works well for the purposes of this assignment. A more advanced system would use a proper head pose estimation with PnP algorithm to get exact rotation angles, but the nose-based method is reliable enough for detecting large lateral head movements.

3.3 Distraction Logic

The system implements two different types of distraction detection:

Long distraction: If the driver continuously looks away from the road for 5 seconds or more, the state changes to "Distracted (long)". This state is cleared only when the driver looks forward for at least 2 consecutive seconds.

Short distraction: To detect repeated brief glances away from the road, the system uses a rolling time window of 30 seconds. It accumulates the total time the driver has

been looking away within this window. If this accumulated time reaches 10 seconds, the state changes to "Distracted (short)". This state is also cleared after 2 seconds of looking forward.

The rolling window is implemented using a Python deque that stores (timestamp, is_away) pairs. Old entries are automatically removed when they fall outside the 30-second window.

Figure: System showing Distracted (long) state

3.4 Eye Closure Detection

The system uses the Eye Aspect Ratio (EAR) method, introduced by Soukupova and Cech in 2016, to measure how open or closed the eyes are. The formula uses 6 landmark points around each eye:

$$EAR = (||p2 - p6|| + ||p3 - p5||) / (2 \times ||p1 - p4||)$$

When the eye is fully open, the EAR value is approximately 0.25 to 0.30. When the eye is closed, it drops close to 0. A threshold of 0.20 was chosen as the decision boundary. If both eyes go below this threshold simultaneously, the system starts counting the closure time.

Two sleep-related states are detected:

- Microsleep: both eyes closed for 4 seconds or more.
- Sleep: both eyes closed for 7 seconds or more.

Both states are cleared only after both eyes remain open for at least 2 seconds. This prevents false recoveries from a single blink.

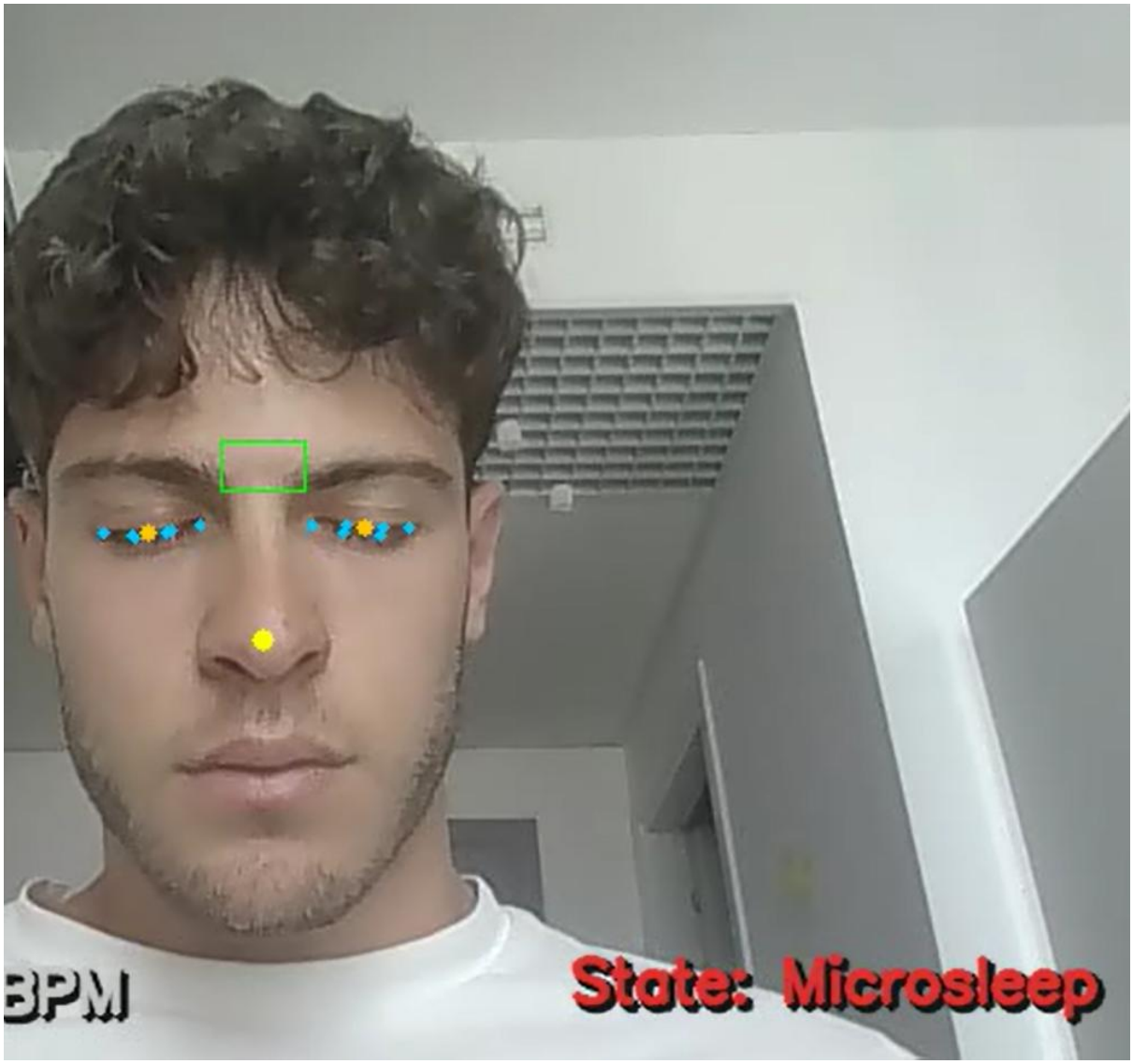

Figure: System showing Microsleep state

3.5 Heart Rate Estimation

The heart rate is estimated using a technique called remote photoplethysmography (rPPG). The idea behind it is that each time the heart beats, a small pulse of blood passes through the capillaries in the skin. This blood absorbs green light, so the

green channel of the camera image over the forehead area changes very slightly with each heartbeat.

The system extracts a region of interest (ROI) from the forehead area each frame and records the mean value of the green channel. After accumulating 8 seconds of signal, it applies the following processing steps:

- Subtract the mean to remove slow brightness changes (detrending).
- Apply a 4th order Butterworth bandpass filter between 0.75 Hz and 3.0 Hz (45 to 180 BPM).
- Apply a Hanning window and compute the FFT.
- Find the dominant frequency in the valid heart rate range and convert to BPM.

The main limitation of this method is that standard webcams apply internal compression and automatic exposure correction, which can mask the very small colour variations caused by the pulse. Good lighting conditions and staying still during estimation significantly improve the results.

Figure: System showing estimated heart rate in BPM

4. Results

The system was tested in a controlled indoor environment with a standard laptop webcam. The five driver states were tested individually and the system correctly identified each one.

4.1 Driver States

The table below summarises all possible states and the conditions that trigger them:

Priority	State	Condition
1 (highest)	Sleep	Both eyes closed >= 7 s
2	Microsleep	Both eyes closed >= 4 s
3	Distracted (long)	Looking away >= 5 s continuously
4	Distracted (short)	Accumulated away time >= 10 s in 30 s window
5 (lowest)	Focused on the road	None of the above conditions met

4.2 System Parameters

The following table shows the final parameter values used in the system and the reason for each choice:

Parameter	Value	Reason
`CALIBRATION_TIME`	3.0 s	Enough time to settle and look forward
`NOSE_THRESHOLD`	0.08	8% of frame width; robust to small movements
`LONG_DISTRACTION_TIME`	5.0 s	Clear danger if away more than 5 seconds
`SHORT_DISTRACTION_WINDOW`	30.0 s	Rolling window to catch repeated glances
`SHORT_DISTRACTION_ACCUM`	10.0 s	10 s away in 30 s window is too much
`RECOVERY_FORWARD_TIME`	2.0 s	Confirms driver has returned attention
`EYE_AR_THRESHOLD`	0.20	Below this EAR value the eye is considered closed
`MICROSLEEP_TIME`	4.0 s	Eyes closed 4 s indicates microsleep risk

Parameter	Value	Reason
SLEEP_TIME	7.0 s	Eyes closed 7 s is a clear sleep event
EYE_OPEN_RECOVERY_TIME	2.0 s	Both eyes open 2 s to confirm wakefulness
HEART_RATE_WINDOW	8.0 s	Minimum signal length for reliable FFT

4.3 Heart Rate

Under good lighting conditions and with the driver remaining still, the system was able to produce a heart rate estimate after approximately 8 seconds of signal collection. The estimated values were compared manually with a reference pulse measurement and showed reasonable accuracy for a webcam-based method, with typical differences of around 5 to 10 BPM.

5. Discussion and Limitations

The system works correctly for the required assignment specifications, but there are some limitations that are worth mentioning.

Gaze estimation: The nose-based approach works well for detecting large lateral head movements, but it cannot detect vertical movements or situations where the driver looks sideways with their eyes without moving their head

Heart rate estimation: The rPPG method is very sensitive to lighting conditions, camera compression, and movement. Most consumer webcams apply automatic exposure and white balance adjustments that interfere with the subtle colour variations caused by the pulse.

Real driving conditions: The system was tested in a static environment. In a real car, the lighting changes constantly and the driver moves more, which would affect all detection modules. A production system would need training data from real driving scenarios and more robust algorithms.

6. Conclusion

This assignment implemented a complete Driver Monitoring System in Python using a standard laptop webcam. The system can detect five driver states in real time — focused, two types of distraction, microsleep, and sleep — and displays an estimated heart rate. All components are implemented in separate, well-documented modules that can be improved or replaced independently.

The project demonstrates that it is possible to build a functional safety monitoring system using only a webcam and open-source libraries, which is relevant for low-cost implementations in vehicles where dedicated hardware is not available.